Tarea 1 - ISIS 4221

Procesamiento de Lenguaje Natural

Fecha de entrega: domingo 23 de agosto, 11:59 p. m. No se permiten entregas tardías; la plataforma se cierra automáticamente.
Modalidad: individual o en grupos de máximo tres estudiantes.
Entrega: un archivo ZIP con los notebooks y el reporte en formato PDF. Solo un integrante del grupo debe realizar la entrega a través de Bloque Neón.

Reglas de codificación

Use Jupyter Notebooks con Python y asegúrese de que cada notebook se ejecute y contenga los resultados antes de enviarlo. Todas las clases, métodos, funciones y código libre deben contener docstrings con una explicación detallada. Para facilitar la calificación, lea todos los archivos suponiendo que se encuentran en la misma carpeta que los notebooks; no use rutas absolutas. Construya un notebook para cada punto.

Las implementaciones de los tres primeros puntos deben ser propias. No se permite el uso de librerías de recuperación de información (`scikit-learn`, `gensim`, `rank_bm25`, `whoosh`, `pyserini`, etc.), salvo donde se indique explícitamente. `nltk` o `spaCy` pueden usarse únicamente para tokenización, listas de palabras vacías y stemming.

El reporte debe incluir las respuestas, las tablas de resultados solicitadas y un breve resumen de la implementación.

Actividades

[10p] Métricas de evaluación

Implemente las siguientes métricas de evaluación de IR usando Python+`numpy` (debe usar `numpy`). Cada función debe manejar los *edge cases* (k mayor que la longitud del vector de relevancia, vectores vacíos y consultas sin documentos relevantes) devolviendo `0.0` en lugar de lanzar una excepción, y debe estar acompañada de al menos dos pruebas propias además del ejemplo dado.

- **[2p] Precision y Precision at K** (la relevancia es binaria).

```
>>> relevance_query = [0, 1, 0, 0, 1]
>>> precision(relevance_query)
0.4
>>> k = 3
>>> precision_at_k(relevance_query, k)
0.3333333
```

- **[1p] Recall at K** (la relevancia es binaria).

```
>>> relevance_query = [0, 1, 0, 0, 1]
>>> number_relevant_docs = 4
>>> k = 3
>>> recall_at_k(relevance_query, number_relevant_docs, k)
```

```
0.25
```

- **[2p] Average precision** (la relevancia es binaria). Suponga que el vector binario de entrada contiene todos los documentos relevantes de la consulta.

```
>>> relevance_query = [1, 0, 1, 1, 0, 0, 1, 0]
>>> average_precision(relevance_query)
0.7470238
```

- **[2p] Mean average precision (MAP)** (la relevancia es binaria). La entrada es una lista de vectores binarios; cada uno representa el resultado de una consulta y contiene todos sus documentos relevantes.

```
>>> relevance_query_1 = [1, 0, 1, 1, 0, 0, 1, 0]
>>> relevance_query_2 = [0, 1, 1, 0, 1]
>>> relevance_queries = [relevance_query_1, relevance_query_2]
>>> mean_average_precision(relevance_queries)
0.6679563
```

- **[2p] DCG at K** (la relevancia es un número natural). La función debe recibir un parámetro que seleccione la ganancia lineal (rel_i) o la ganancia exponencial ($2^{\text{rel}_i} - 1$); en ambos casos use el descuento $\log_2(i+1)$, con i como la posición empezando en 1.

```
>>> relevance_query = [3, 2, 3, 0, 1, 2, 3, 0, 0, 1]
>>> k = 5
>>> dcg_at_k(relevance_query, k, gain='linear')
6.1487123
>>> dcg_at_k(relevance_query, k, gain='exponential')
12.7796420
```

- **[1p] NDCG at K** (la relevancia es un número natural).

```
>>> relevance_query = [3, 2, 3, 0, 1, 2, 3, 0, 0, 1]
>>> k = 5
>>> ndcg_at_k(relevance_query, k, gain='linear')
0.7659228
>>> ndcg_at_k(relevance_query, k, gain='exponential')
0.7357689
```

Comparación de estrategias de motores de búsqueda

A continuación, implementará un motor de búsqueda con tres estrategias diferentes.

1. Recuperación booleana usando índice invertido (BSII).
2. Recuperación ranqueada y vectorización de documentos (RRDV).
3. Recuperación ranqueada con GENSIM.

Debe hacer su propia implementación usando `numpy` y `pandas` para los dos primeros puntos.

Conjunto de datos: hay tres archivos que componen el conjunto de datos.

- "Docs raws texts" contiene 331 documentos en formato NAF (XML; debe usar el título y el contenido para modelar cada documento).

- "Queries raw texts" contiene 35 consultas.
- `relevance-judgments.tsv` contiene para cada consulta los documentos considerados relevantes. Estos documentos fueron seleccionados manualmente por jueces humanos y sirven como *ground truth* para la evaluación.

Pasos de preprocesamiento: para los siguientes puntos, debe preprocesar documentos y consultas mediante tokenización a nivel de palabra, eliminación de palabras vacías, normalización y stemming. Incluya en el informe una tabla con el tamaño del vocabulario y el número de tokens después de cada paso, y justifique las decisiones sobre números, fechas, palabras con guion y términos acentuados. El mismo preprocesamiento debe aplicarse a las tres estrategias.

[30p] Recuperación booleana usando índice invertido (BSII)

[10p] Cree su propia implementación del índice invertido usando los 331 documentos en el conjunto de datos. Los *skip pointers* son atajos entre elementos no consecutivos de una lista de postings ordenada. Durante una intersección permiten omitir elementos que no pueden pertenecer al resultado cuando el destino del salto no supera el documento actual de la otra lista. Para una lista de postings de longitud P, use $\sqrt{P}$ *skip pointers* espaciados uniformemente. Reporte el tamaño en memoria del índice y el número de postings del término más frecuente y del menos frecuente.

[15p] Cree una función que lea el índice invertido y calcule consultas booleanas mediante el algoritmo de mezcla. El algoritmo debe calcular AND, OR, NOT, respetar la precedencia indicada con paréntesis y usar los *skip pointers* durante la intersección.

La entrada de la función es un string de la forma `early AND telecommunication AND methods`.

Responda: ¿cuál es la complejidad del algoritmo de mezcla con y sin *skip pointers*? ¿Por qué los *skip pointers* no son útiles para resolver consultas OR? ¿Cómo se compara el algoritmo de mezcla con y sin *skip pointers* para la consulta `early AND telecommunication`?

[5p] Para cada una de las 35 consultas en el conjunto de datos, recupere los documentos utilizando consultas binarias AND (`termino_1 AND termino_2 AND termino_3...`). Escriba los resultados en:

`BSII-AND-queries_results`

Use el mismo formato que `relevance-judgments`:

```
q01 dXX,dYY,dZZ...
```

Nota: el archivo puede estar vacío.

[30p] Recuperación ranqueada y vectorización de documentos (RRDV)

[10p] Cree una función que, a partir del índice invertido, cree la representación vectorial ponderada tf.idf de un documento o consulta. Describa en detalle su estrategia: ¿es eficiente?, ¿por qué sí o por qué no?

[10p] Cree una función que reciba dos vectores de documentos y calcule la similitud del coseno. Use además BM25 como segunda función de ranqueo; para BM25 puede utilizar una librería que ya

implemente el algoritmo. Deje k_1 y b como parámetros. Explique la diferencia entre la normalización por longitud del coseno y la que hace BM25, y reporte cómo cambian los resultados al variar b entre 0 y 1.

[5p] Para cada una de las 35 consultas en el conjunto de datos, recupere los documentos clasificados, ordenados por el puntaje de similitud. Incluya solo los documentos con un puntaje superior a 0 para una consulta determinada. Escriba un archivo por función de ranqueo (`RRDV-consultas_resultados` y `BM25-consultas_resultados`) con los resultados siguiendo el formato:

```
q01 dXX: simi(q01,dXX),dYY: simi(q01,dYY),...
```

[5p] Evaluación de resultados. Calcule P@M, R@M y NDCG@M por consulta. M es el número de documentos relevantes encontrados en el archivo de juicios de relevancia por consulta. Luego calcule MAP como métrica general, reutilizando las funciones del primer punto.

Nota I: para P@M y R@M suponga una escala de relevancia binaria. Los documentos que no se encuentran en el archivo `relevance-judgments` no son relevantes para una consulta determinada.

Nota II: para NDCG@M utilice la escala de relevancia no binaria que se encuentra en el archivo `relevance-judgments`.

[30p] Recuperación ranqueada con GENSIM

Repita la recuperación ranqueada con tf.idf del punto anterior utilizando GENSIM. Use únicamente similitud coseno como función de ranqueo. Use el siguiente nombre:

`GENSIM-consultas_resultados`

Evalúe los resultados con P@M, R@M, NDCG@M y MAP.